

Minesweeper architecture

1. User stories

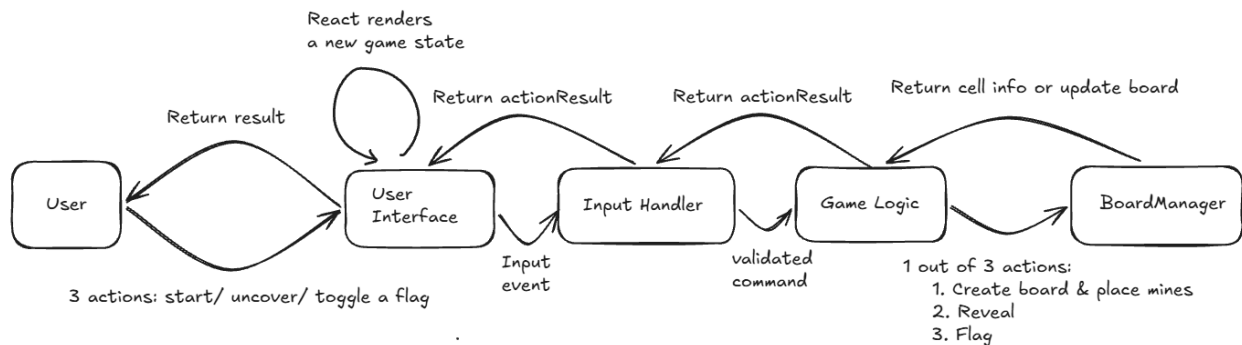
- As a player, I want to choose the mine count so I can adjust difficulty.
- As a player, I want to reveal cells and read nearby-mine counts so I can choose my next move.
- As a player, I want to flag and unflag suspected mines so I can track my guesses.
- As a player, I want clear win/loss feedback and a restart option so I can play another round.

2. Scope and rules

- Single-player React + TypeScript browser game: fixed 10×10 board and 10–20 mines.
- Includes a safe first reveal, automatic empty-region expansion, flags, remaining-flag counter, and restart.
- Reveal all safe cells to win; reveal a mine after the protected first move to lose.
- Flags are capped at the chosen mine count.
- Flagged cells must be unflagged before revealing them.
- Finished games ignore further board actions.

3. Overall architecture

Four modules run inside one browser app. The UI stores the game state; the other modules receive it and return results.



Game Logic returns an `ActionResult` through `Input Handler`. The UI saves a changed state and redraws the screen. Invalid input shows an error; an ignored action keeps the same state.

For the empty setup preview, the UI calls `Board Manager` directly.

4. Components and data structures

Components

- **User Interface** (`src/UserInterface.tsx`): stores the current game, shows the screen, and sends player actions to `Input Handler`. It also stores errors and the clicked mine highlight.

- **Input Handler** (src/inputHandler.ts): checks mine-count text and cell positions, then calls Game Logic. It can parse A1–J10 references; the current UI uses cell buttons.
- **Game Logic** (src/gameLogic.ts): applies the rules for starting, revealing, flagging, and winning or losing. Its three public commands are startGame(mineCount, random?), uncover(state, position, random?), and toggleFlag(state, position).
- **Board Manager** (src/boardManager.ts): creates and updates the grid, places or moves mines, finds neighbors, and calculates nearby-mine counts.

Data structures : defined in types.ts

- **Position:** row and column numbers from 0–9. The screen labels them 1–10 and A–J.
- **Cell:** hasMine says whether a mine is present; adjacentMines is the nearby-mine count (0–8); visibility is covered, flagged, or revealed.
- **Board:** a 10×10 array of cells, accessed as board[row][column].
- **GameState holds the current round:**
 - board: the grid.
 - mineCount: chosen mine total, from 10–20.
 - flagsPlaced: number of flags placed; starts at 0.
 - revealedSafeCount: number of safe cells opened; starts at 0.
 - firstRevealDone: whether first-reveal protection has been used; starts false.
 - status: playing, won, or lost; starts playing. Before a game starts, the UI stores null instead of a GameState.
- **ActionResult is what a command returns:**
 - **Changed:** {ok: true, changed: true, state} : the UI stores the returned state.
 - **Ignored:** {ok: true, changed: false, state} : the original state stays in place.
 - **Invalid:** {ok: false, code, message} how the error. Codes are INVALID_MINE_COUNT and INVALID_POSITION.
- **RandomSource:** an optional function returning a finite number from 0 up to, but not including, 1. It defaults to Math.random; tests can supply a predictable source.

Board helpers

- **Creation:** createBoard and createBoardWithMines.
- **Access:** isValidPosition, getCell, updateCell, and getNeighbors. Neighbors include diagonals.
- **Mines:** calculateAdjacentMines, makeFirstCellSafe, and exposeAllMines.
- updateCell does not refresh nearby-mine counts; placement and relocation do. Invalid board access, configuration, or random values can throw RangeError.

State notes

The types are read-only, and updates preserve the previous state. Mine locations exist in the state but stay hidden on covered cells.

“Mines left” = mineCount – flagsPlaced. After a loss, flagsPlaced keeps its pre-loss value even when flagged mines become visible.

5. Main action flows

Start a round

1. SetupControls sends the chosen count through the UI to processSetupInput.

2. startGame validates 10–20. createBoardWithMines places distinct mines and calculates nearby counts.
3. The UI stores the new state: all cells covered, both counters at 0, status playing, and firstRevealDone false.

Reveal a cell

1. GameBoard sends the selected position through the UI to processUncoverInput. uncover ignores finished games and flagged or already revealed cells.
2. On the first eligible reveal, makeFirstCellSafe moves a clicked mine to a random covered, safe, unflagged cell and updates nearby counts. Mine total stays the same; ignored actions do not use this protection.
3. If the cell contains a mine, loseGame calls exposeAllMines and ends the round. Otherwise, revealSafeArea opens the safe cell and expands connected empty areas and their bordering numbers.
4. After a safe reveal, withWinStatus checks whether all $100 - \text{mineCount}$ safe cells are open. Input Handler returns the result; the UI stores and displays it.

Flag or unflag a cell

1. GameBoard sends the position through the UI to processFlagInput.
2. toggleFlag switches covered ↔ flagged and updates flagsPlaced. Revealed cells and finished games do not change.
3. The UI redraws the cell and counter. At the flag limit, only unflagging is allowed. Flags alone cannot win the game.